GUILHOT Benjamin

VAN DOORNIK Levi

Advanced Topics Reflection — BenLeViRPG

Topic 1: Unit Testing with Junit

Why we chose this topic ?

Our game contains a significant amount of logic that is hard to verify by playing alone: damage formulas, type multipliers, critical hits, XP gain, and level-up stat increases all interact with each other. A bug in any of these can break the entire game loop without being immediately obvious. We chose unit testing with JUnit to have a reliable way to verify each piece of logic independently, and to catch regressions whenever we changed something.

What we learned ?

We learned how JUnit 5 works: the @Test annotation, assertion methods such as assertEquals and assertTrue, and how to structure test classes that are independent of each other. The most important concept we learned was how to write deterministic tests for code that involves randomness. Our damage formula includes a critical hit multiplier drawn from a random float. To make tests reproducible, we pass a crit chance of 0.0f (never crits) or 1.0f (always crits) directly to the Fighter constructor, so the outcome is always the same and assertions can use exact expected values.

How we applied it ?

We applied unit testing throughout the development of the game, writing tests for every core component: damage calculation, healing, level-up, XP gain, win and lose conditions, and special attacks. Rather than testing implementation details, we focused on observable behaviour — given a fighter with specific stats, what damage should result? Given a hero with a certain amount of XP, does it level up at the right threshold? This approach meant that tests remained valid even when we refactored internals.

What we learned from the implementation ?

Writing tests before or alongside features forced us to think about edge cases we would have missed otherwise, such as what happens when a hero's attack is lower than the enemy's defense (damage must be at least 1), or when a hero is much stronger than an enemy (XP should not be zero). Tests also revealed a real balance issue: when we changed the Warrior's ATK value, several tests broke immediately, which helped us update the expected values consistently across all files.
Without tests, those inconsistencies would have gone unnoticed.

Topic 2: UX & Game Design

Why we chose this topic ?

An RPG where all characters feel identical and fights are decided by luck is not fun. We chose UX and game design to think systematically about what makes the game playable and engaging: meaningful character differences, a readable interface, and a combat system balanced enough that the player can actually win.

What we learned ?

We learned that game balance is not intuitive and requires testing with concrete numbers. A type system adds strategic depth only if the multipliers are tuned correctly: too strong and type counters make fights trivial, too weak and the system has no impact. We also learned about visual feedback in interfaces: players need to understand what is happening at a glance. Color, layout, and text all contribute to readability, playability and identity of the game.

How we applied it ?

We designed a type system with four types (Normal, Fire, Water, Thunder) forming a triangle of strengths and weaknesses. After testing, we settled on multipliers of 1.3 for a type advantage and 0.8 for a disadvantage, which create a noticeable but not overwhelming effect. For the interface, we replaced the default Swing look with a dark warm palette (deep brown background, gold buttons) that matches the fantasy theme of the game. Type names are rendered in color using HTML labels inside Swing components. Character sprites are loaded from resources and displayed in both the selection screen and the battle screen. A StatusBox component shows each fighter's HP bar, level, and XP in a compact, readable format. We also added special attack and personal statistics to each hero. To finish, we added a coin system when beating an enemy that allow the player to buy statistics upgrade.

What we learned from the implementation ?

The most important lesson was that balance problems only appear when you play the game. Early in development, the Warrior had an ATK of 5 against a Fire-type enemy with a type advantage, which reduced the effective damage to 1 per hit, making the fight nearly unwinnable. We identified this by analysing the damage formula with actual stat values and fixed it by raising the Warrior's ATK and softening the type multipliers. On the UI side, we discovered that Swing's default button rendering conflicts with custom background colors on some platforms unless setOpaque(true) and setBorderPainted(false) are set explicitly. These small details took time to get right but had a large impact on how the final application looks.